Programación Orientada a Objetos

El torneo cósmico en el que Java te obliga a invocar public static void main como un ataque especial, mientras lloras entre errores de sintaxis que se multiplican como espectros en el Santuario

Daniel Barajas Higuera



Contenido

1. POO
2. SS
3. Abstracción
4. Encapsulamiento
5. Herencia
6. Polimorfismo
7. Composición
8. Delegación

Introducción a la POO

La **programación orientada a objetos** surge en los años 60 con *Simula*, pensada para modelar sistemas complejos, y se populariza en los 80 con lenguajes como *Smalltalk* y *C++*. Su propósito es representar entidades del mundo real en el código, integrando datos y acciones en una misma unidad llamada **objeto**.

Conceptos de POO

1. **Abstracción**: centrarse en los rasgos esenciales de un objeto.
2. **Encapsulamiento**: proteger los datos internos y controlar su acceso.
3. **Herencia**: permitir que una clase derive de otra y reutilice su código.
4. **Polimorfismo**: redefinir o variar comportamientos manteniendo la misma interfaz.

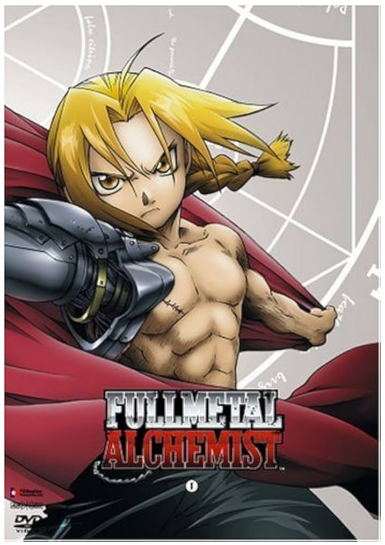
Pilares



Lenguajes que usan Programación Orientada a Objetos (I)



Alquimia peligrosa: usas `templates` y terminas invocando horrores del compilador.



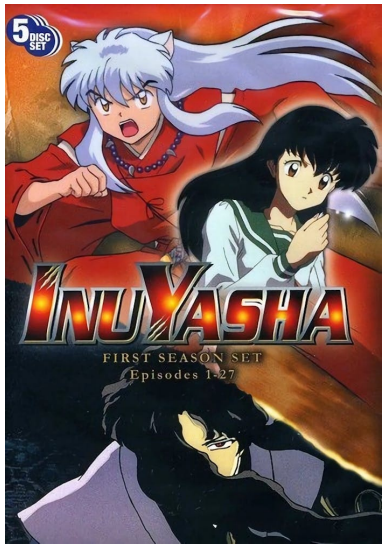


Colorido y brillante, con transformaciones mágicas repetidas una y otra vez.





Viejo y parchado: nadie lo mata, sigue apareciendo.





Tan verboso como un discurso de Atena. Métodos con 20 líneas y puro *boilerplate*.



Lenguajes que usan Programación Orientada a Objetos (II)



Un caos incoherente que funciona por milagro; imposible de matar.



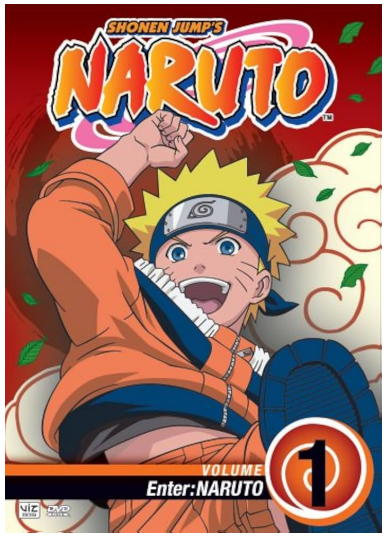


Vampiro creado por Microsoft, te chupa la vida con su framework y nunca muere.





Fácil como un jutsu de academia, pero usado para todo, incluso lo absurdo.





Apple decidiendo qué puedes y no puedes hacer en tu app como si fueran el Sistema Sibyl.



Contenido

1 POO

2 SS

3 Abstracción

4 Encapsulamiento

5 Herencia

6 Polimorfismo

7 Composición

8 Delegación

Java × Saint Seiya





- Siempre el mismo arco: verbose, ritual, pero aguanta todo.
- Caballeros con armaduras = frameworks infinitos.
- JVM es el cosmos que sostiene a todos.
- Heroico, repetitivo y aún corporativamente invencible.

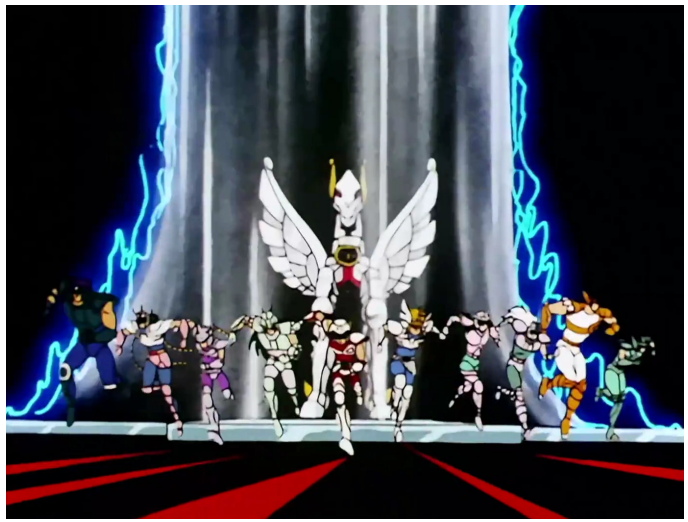
Saint Seiya

Saint Seiya es un manga y anime creado por Masami Kurumada en 1986. Narra la historia de jóvenes caballeros que protegen a Atenea canalizando su **cosmos** y portando **armaduras** asociadas a constelaciones.

- **Anime**: sagas como las Doce Casas, Asgard, Poseidón y Hades.
- **Manga**: la obra original y precuelas como Lost Canvas.
- **Merchandising**: figuras, videojuegos, cartas.

Saint Seiya: introducción

- Historia de jóvenes caballeros que protegen a Atenea.
- El eje es su crecimiento, elevando el cosmos para superar enemigos cada vez más poderosos.
- Cada caballero está vinculado a una constelación y su armadura refleja ese lazo.



Atenea, caballeros y cosmos

- **Caballeros:** guerreros con armaduras sagradas, ligadas a las constelaciones.
- **Cosmos:** energía vital que cada uno aprende a encender y expandir; determina la fuerza de sus ataques y defensas.
- El cosmos representa el límite a superar en cada batalla: cuanto más se domina, mayor es el poder despertado.



Rangos y jerarquías

- **Bronce:** los protagonistas, caballeros iniciales, luchan desde la base.
- **Plata:** nivel intermedio, guardianes con técnicas más refinadas.
- **Oro:** los 12 caballeros más poderosos, guardianes de las Doce Casas del Zodiaco.
- La jerarquía no solo es de poder, también de responsabilidad: cada rango protege un nivel del santuario.



Armaduras y evolución

- **Armaduras:** protegen, amplifican cosmos y marcan identidad del caballero.
- Cada armadura puede ser dañada y reparada, incluso con la sangre de su portador.
- **Evolución:** al elevar el cosmos, las armaduras alcanzan nuevas formas (divinas o de dioses).
- El vínculo caballero–armadura es central: sin ella, pierde poder; con ella, alcanza su máximo potencial.



Contenido

1. POO
2. SS
3. Abstracción
   - Clases
   - Objetos
   - Atributos
   - Métodos
4. Encapsulamiento
5. Herencia
6. Polimorfismo
7. Composición
8. Delegación

Abstracción

Es el proceso de identificar las características y comportamientos esenciales de un objeto del mundo real y convertirlos en una representación simplificada y manejable en el código.

La abstracción define las características esenciales del caballero: `constelacion`, `rango`, `cosmos`, `armadura`, `ataque`, `defensa`, `velocidad`.



Clase Caballero (atributos)

```
public class Caballero {
    // Atributos
    private final String nombre;
    private final String constelacion;
    private final String rango;

    private int hp, atk, pDef, mDef;
    private int hit, eva, crt, crtRes, flee, breakStat;
    private int hpRecovery, xpRecovery;

    // Constructor irá en el siguiente frame
}
```

Clase Caballero (constructor)

```
public class Caballero {
    // ... atributos

    // Constructor de la clase
    public Caballero(String nombre, String constelacion, String rango,
                     int hp, int atk, int pDef, int mDef,
                     int hit, int eva, int crt, int crtRes,
                     int flee, int breakStat,
                     int hpRecovery, int xpRecovery) {
        this.nombre = nombre;
        this.constelacion = constelacion;
        this.rango = rango;
        this.hp = hp; this.atk = atk;
        this.pDef = pDef; this.mDef = mDef;
        this.hit = hit; this.eva = eva;
        this.crt = crt; this.crtRes = crtRes;
        this.flee = flee; this.breakStat = breakStat;
        this.hpRecovery = hpRecovery;
        this.xpRecovery = xpRecovery;
    }
}
```

Instanciación: Shaka de Virgo

Un objeto es un caballero específico: Seiya, Shiryū, Hyōga, Shaka.

```
// Datos tomados de las estadísticas iniciales
Caballero shaka = new Caballero(
    "Shaka", "Virgo", "Oro",
    350, 153, 38, 49, // HP, ATK, P-DEF, M-DEF
    24, 20, 24, 20,   // HIT, EVA, CRT, CRT.RES
    16, 16,           // FLEE, BREAK
    50, 120           // HP Recovery, XP Recovery
);
```



Atributos

```
System.out.println(shaka.getNombre());
System.out.println(shaka.getConstelacion());
System.out.println(shaka.getRango());
System.out.println(shaka.getHp());
System.out.println(shaka.getAtk());
System.out.println(shaka.getMDef());
```



Métodos

En este punto, los métodos representan técnicas que el caballero puede ejecutar. Definimos dos para Shaka de Virgo:

- **Tesoro del Cielo**: ataque espiritual concentrado.
- **Tenbu Hōrin**: técnica que limita al oponente.



Métodos dentro de Caballero

```java
// Dentro de la clase Caballero (mismo archivo)
// Añadimos métodos sin usar herencia
public void tesoroDelCielo() {
    System.out.println(nombre + " ejecuta Tesoro del Cielo!");
    // Ejemplo: subir momentáneamente precisión y crítico
    hit = Math.min(999, hit + 5);
    crt = Math.min(999, crt + 5);
}

public void tenbuHorin() {
    System.out.println(nombre + " desata Tenbu Hōrin y limita al oponente.");
    // Ejemplo: autorregulación breve del cosmos mediante recuperación
    hpRecovery = Math.min(999, hpRecovery + 10);
}
```

Uso de métodos

```
// Instancia existente (Shaka)
// y llamadas a métodos

Caballero shaka = new Caballero(
    "Shaka", "Virgo", "Oro",
    350, 153, 38, 49,
    24, 20, 24, 20,
    16, 16,
    50, 120
);

shaka.tesoroDelCielo();
shaka.tenbuHorin();
```



Contenido

1. POO
2. SS
3. Abstracción
4. Encapsulamiento
5. Herencia
6. Polimorfismo
7. Composición
8. Delegación

Encapsulamiento

Así como la urna protege la armadura, Shaka protege sus técnicas. No cualquiera puede acceder a su máximo poder.

Esto refleja el encapsulamiento en POO:

- Protección de técnicas internas (atributos privados).
- Acceso solo mediante métodos definidos (interfaz pública).
- Garantiza coherencia y control.



Ejemplo en código (Java): Shaka de Virgo

```java
public class ShakaDeVirgo {
    private boolean ojosAbiertos;

    public ShakaDeVirgo() {
        this.ojosAbiertos = false;
    }

    // Método público
    public void usarTecnica(String tecnica) {
        if (tecnica.equals("Tenbu Horin")) {
            System.out.println("Shaka ejecuta Tenbu Hōrin.");
        } else if (tecnica.equals("Abrir Ojos") && !ojosAbiertos) {
            ojosAbiertos = true;
            System.out.println("Shaka abre los ojos: poder oculto liberado.");
        } else {
            System.out.println("Técnica no accesible.");
        }
    }
}
```

Ejemplo de ejecución

```java
public class DemoShaka {
    public static void main(String[] args) {
        ShakaDeVirgo shaka = new ShakaDeVirgo();
        // acceso público
        shaka.usarTecnica("Tenbu Horin");
        // acceso restringido
        shaka.usarTecnica("Abrir Ojos");
        // no permitido
        shaka.usarTecnica("Explosión Galáctica");
    }
}
```

```
Shaka ejecuta Tenbu Hōrin.
Shaka abre los ojos: poder oculto liberado.
Técnica no accesible.
```



Contenido

1. POO
2. SS
3. Abstracción
4. Encapsulamiento
5. Herencia
6. Polimorfismo
7. Composición
8. Delegación

Herencia en POO

La **herencia** permite que una clase (*subclase*) aproveche atributos y métodos de otra clase (*superclase*).

- Facilita la **reutilización de código**: no se repite lo común.
- Permite **extender o modificar comportamiento**: cada subclase añade lo propio.
- Establece **jerarquías naturales** entre clases.

Herencia en Saint Seiya: Bronce $\rightarrow$ Plata $\rightarrow$ Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.

Herencia en Saint Seiya: Bronce $\rightarrow$ Plata $\rightarrow$ Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.
- **Caballeros de Plata**: heredan lo de los de Bronce, pero con técnicas y armaduras más avanzadas.

Herencia en Saint Seiya: Bronce $\rightarrow$ Plata $\rightarrow$ Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.
- **Caballeros de Plata**: heredan lo de los de Bronce, pero con técnicas y armaduras más avanzadas.
- **Caballeros de Oro**: en la cima de la jerarquía, heredan la estructura pero amplían al máximo poder y defensa.

Herencia en Saint Seiya: Bronce → Plata → Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.
- **Caballeros de Plata**: heredan lo de los de Bronce, pero con técnicas y armaduras más avanzadas.
- **Caballeros de Oro**: en la cima de la jerarquía, heredan la estructura pero amplían al máximo poder y defensa.



Armadura de Pegaso (Bronce)

Herencia en Saint Seiya: Bronce → Plata → Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.
- **Caballeros de Plata**: heredan lo de los de Bronce, pero con técnicas y armaduras más avanzadas.
- **Caballeros de Oro**: en la cima de la jerarquía, heredan la estructura pero amplían al máximo poder y defensa.



Armadura de Pegaso (Bronce)



Armadura de Lagarto (Plata)

Herencia en Saint Seiya: Bronce → Plata → Oro

En el Santuario, los caballeros comparten una misma base: son guerreros que visten armaduras y elevan su cosmos.

- **Caballeros de Bronce**: la clase más básica, con atributos modestos.
- **Caballeros de Plata**: heredan lo de los de Bronce, pero con técnicas y armaduras más avanzadas.
- **Caballeros de Oro**: en la cima de la jerarquía, heredan la estructura pero amplían al máximo poder y defensa.



Armadura de Pegaso (Bronce)



Armadura de Lagarto (Plata)



Armadura de Géminis (Oro)

Herencia en Java: clase base

```
class Caballero {
    protected String nombre, constelacion, rango;
    protected int ataque, defensa;

    Caballero(String n, String c, String r, int atk, int def) {
        nombre = n; constelacion = c; rango = r;
        ataque = atk; defensa = def;
    }

    public String ataqueEspecial() {
        return nombre + " concentra cosmos básico.";
    }
}
```

Herencia en Java: rangos

```
class CaballeroBronce extends Caballero {
    CaballeroBronce(String n, String c) { super(n,c,"Bronce",40,30); }
    @Override public String ataqueEspecial() {
        return nombre + " lanza técnica de Bronce.";
    }
}

class CaballeroPlata extends Caballero {
    CaballeroPlata(String n, String c) { super(n,c,"Plata",55,45); }
    @Override public String ataqueEspecial() {
        return nombre + " ejecuta arte de Plata.";
    }
}

class CaballeroOro extends Caballero {
    CaballeroOro(String n, String c) { super(n,c,"Oro",80,70); }
    @Override public String ataqueEspecial() {
        return nombre + " desata el séptimo sentido.";
    }
}
```

Uso de la jerarquía de Caballeros

```java
public class DemoHerencia {
    public static void main(String[] args) {
        CaballeroBronce seiya = new CaballeroBronce("Seiya","Pegaso");
        CaballeroPlata misty = new CaballeroPlata("Misty","Lagarto");
        CaballeroOro saga = new CaballeroOro("Saga","Géminis");

        System.out.println(seiya.ataqueEspecial());
        System.out.println(misty.ataqueEspecial());
        System.out.println(saga.ataqueEspecial());
    }
}
```

```
Seiya lanza técnica de Bronce.
Misty ejecuta arte de Plata.
Saga desata el séptimo sentido.
```

Contenido

1 POO

2 SS

3 Abstracción

4 Encapsulamiento

5 Herencia

6 Polimorfismo

7 Composición

8 Delegación

¿Qué es polimorfismo?

- Un **mismo mensaje** (misma interfaz/método) produce **comportamientos distintos** según el objeto que lo recibe.
- Permite escribir código que **no conoce** la implementación concreta, solo la interfaz.
- En cristiano: llamas `ataqueEspecial()` y cada quien responde a su estilo sin romper el programa.



Analogía en el Kurumada-verse

Kurumada recicla el *molde Seiya* en protagonistas de distintos animes. Mismo "molde" narrativo, **respuestas distintas**:

- **Seiya** (Saint Seiya)
- **Ryuji Takane** (Ring ni Kakero)
- **Kojirō** (Fūma no Kojirō)
- **Teppei Takamiya** (B't X)

Mismo mensaje: `ataqueEspecial()`. Cada protagonista implementa su propia versión.





Ejemplo en código (Java): interfaz + primeras clases

```java
public interface Protagonista {
    String nombre();
    String ataqueEspecial(); // misma firma, distintas respuestas
}

public class Seiya implements Protagonista {
    public String nombre() { return "Seiya (Saint Seiya)"; }
    public String ataqueEspecial() { return "Meteoros de Pegaso"; }
}

public class Ryuji implements Protagonista {
    public String nombre() { return "Ryuji (Ring ni Kakero)"; }
    public String ataqueEspecial() { return "Uppercut decisivo"; }
}
```

Ejemplo en código (Java): más implementaciones

```java
public class Kojirou implements Protagonista {
    public String nombre() { return "Kojirō (Fūma no Kojirō)"; }
    public String ataqueEspecial() { return "Técnica ninja Fūma"; }
}

public class Teppei implements Protagonista {
    public String nombre() { return "Teppei (B't X)"; }
    public String ataqueEspecial() { return "Carga con B't X"; }
}
```

Ejemplo en código (Java): uso polimórfico

```java
import java.util.List;

public class DemoPolimorfismo {
    public static void main(String[] args) {
        List<Protagonista> equipo = List.of(
        new Seiya(), new Ryuji(), new Kojirou(), new Teppei()
        );

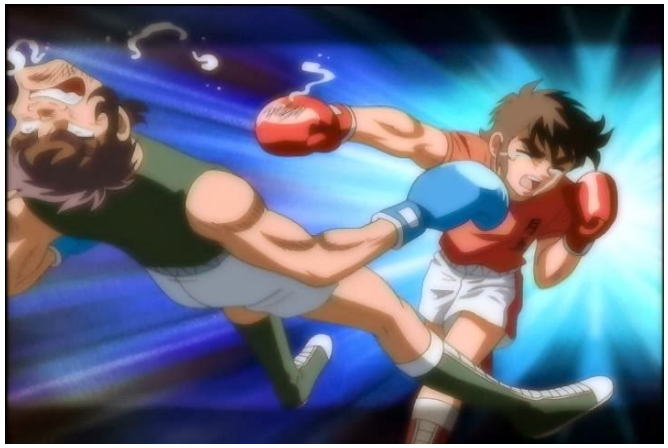
        for (Protagonista p : equipo) {
            System.out.println(p.nombre() + " → " + p.ataqueEspecial());
        }
    }
}
```

```
Seiya (Saint Seiya) → Meteoros de Pegaso
Ryuji (Ring ni Kakero) → Uppercut decisivo
Kojirō (Fūma no Kojirō) → Técnica ninja Fūma
Teppei (B't X) → Carga con B't X
```

Polimorfismo con los 4 protagonistas



Seiya: Meteoros de Pegaso.



Ryuji: gancho/uppercut decisivo en el ring.



Kojirō: técnica ninja con katana.



Teppei: ataque tecno-orgánico.

Contenido

1 POO

2 SS

3 Abstracción

4 Encapsulamiento

5 Herencia

6 Polimorfismo

7 Composición

8 Delegación

Composición en POO

En lugar de heredar jerarquías rígidas, los objetos se construyen **uniendo piezas**.

- Relación “tiene-un”, no “es-un”.
- Cada objeto incorpora colaboradores internos.
- Favorece flexibilidad frente a herencia profunda.

Ejemplo en Saint Seiya

- En la saga de Poseidón, los Caballeros de Bronce siguen siendo ellos mismos.

Ejemplo en Saint Seiya

- En la saga de Poseidón, los Caballeros de Bronce siguen siendo ellos mismos.
- Para destruir los pilares, **usan las armas de Libra**.

Ejemplo en Saint Seiya

- En la saga de Poseidón, los Caballeros de Bronce siguen siendo ellos mismos.
- Para destruir los pilares, **usan las armas de Libra**.
- No se convierten en subclases de Libra: **se componen** con esas armas temporalmente.

Ejemplo en Saint Seiya

- En la saga de Poseidón, los Caballeros de Bronce siguen siendo ellos mismos.
- Para destruir los pilares, **usan las armas de Libra**.
- No se convierten en subclases de Libra: **se componen** con esas armas temporalmente.



Ejemplo en Saint Seiya

- En la saga de Poseidón, los Caballeros de Bronce siguen siendo ellos mismos.
- Para destruir los pilares, **usan las armas de Libra**.
- No se convierten en subclases de Libra: **se componen** con esas armas temporalmente.





Ventajas de y comparación con herencia

- Flexibilidad: se pueden intercambiar componentes sin cambiar la clase principal.
- Reutilización: las mismas armas de Libra sirven a varios caballeros.
- Bajo acoplamiento: cada pieza mantiene su propia lógica.

Herencia (12 Casas)

- Bronce $\rightarrow$ Plata $\rightarrow$ Oro.
- Estructura rígida.
- Difícil de extender.

Composición (Poseidón)

- Caballeros + Armas de Libra.
- Estructura flexible.
- Fácil de reutilizar.

Contenido

1. POO
2. SS
3. Abstracción
4. Encapsulamiento
5. Herencia
6. Polimorfismo
7. Composición
8. Delegación

Delegación en POO

La delegación ocurre cuando un objeto **no ejecuta la tarea por sí mismo**, sino que la **encarga a un colaborador interno**.

- El objeto principal mantiene la interfaz.
- La lógica real la hace otro objeto.
- Se basa en composición, pero con responsabilidad transferida.



Ejemplo en Saint Seiya

- En la saga de Asgard, Syd de Mizar es el Caballero reconocido.
- Sin embargo, muchas acciones son realizadas por Bud en las sombras.
- Para el exterior, **parece que Syd pelea**, pero en realidad está **delegando en Bud**.



Ejemplo en código (Java): Delegación

```java
class Bud {
    public void atacar() {
        System.out.println("Bud ataca desde las sombras.");
    }
}

class Syd {
    private Bud bud;  // colaborador interno

    public Syd() {
        this.bud = new Bud();
    }

    // método público: parece que Syd ataca
    public void atacar() {
        System.out.println("Syd inicia el ataque...");
        bud.atacar(); // delegación real
    }
}

public class DemoDelegacion {
    public static void main(String[] args) {
        Syd syd = new Syd();
        syd.atacar();
    }
}
```

Herencia, Composición y Delegación

Aspecto	Ejemplo Saint Seiya	Ventajas	Desventajas
Herencia	Jerarquía del Santuario (Bronce $\rightarrow$ Plata $\rightarrow$ Oro)	Reutilización estructurada Jerarquía clara	Rigidez Difícil de extender
Composición	Caballeros en la saga de Poseidón usando armas de Libra	Flexibilidad Piezas intercambiables	Coordinación más compleja
Delegación	Syd aparenta luchar, pero Bud ejecuta por él	Encapsula variación Reparte responsabilidades	Oculta quién ejecuta realmente la acción

Programación Orientada a Objetos

La POO nos da las armas conceptuales para organizar el código:

- **Abstracción**: definir lo esencial (constelación, cosmos, técnicas).
- **Encapsulamiento**: como Shaka de Virgo, que mantiene sus ojos cerrados y solo revela su poder en condiciones especiales.
- **Herencia**: jerarquía natural, desde Bronce hasta Oro.
- **Polimorfismo**: un mismo método, múltiples técnicas, como los protagonistas de Kurumada ejecutando su "especial".
- **Composición**: unir piezas, como los caballeros usando las armas de Libra en la saga de Poseidón.
- **Delegación**: encargar tareas a un aliado, como Syd dejando que Bud ejecute en su lugar.

Fin


声の出演
坂田銀時　杉田智和
志村新八　阪口大助
神楽　釘宮理恵
きらめく星座がおまえを呼んでる


原画
佐々木洋平
動画
Wish
一条望
伊藤優希
今西麻綾
ギリシャ神話のように